DEEP REINFORCEMENT LEARNING 365.382 (SS26)

Policy Gradient Methods II: Modern Actor-Critic Algorithms

Sohvi Luukkonen
LIT AI LAB
Institute for Machine Learning

JKU
JOHANNES KEPLER
UNIVERSITY LINZ

Resources

- Reinforcement Learning: An Introduction [Chapters 13–14]
- OpenAI Spinning Up [Policy gradients, TRPO/PPO, DDPG, TD3, SAC]
- TRPO, PPO, SAC

Policy Gradient Methods

RL Taxonomy

- Value-based: learn $q(s, a)$ or $v(s)$, derive policy indirectly
- Policy-based: optimize policy parameters θ directly
- Actor-Critic: learn both policy (actor) and value (critic)

- Last lecture: policy gradients, REINFORCE, AC basics
- This lecture: continuous control and modern actor-critic variants

Motivation for Policy-Based Methods

- True objective: directly optimize behavior policy
- Natural fit for stochastic policies
- Works well with continuous actions

- Useful when value approximation is hard but good policy class is known
- Supports constraints, entropy bonuses, and trust-region updates

Policy Objective (Episodic)

For trajectories $\tau = (s_0, a_0, r_1, \ldots, s_T)$ generated by policy π_θ and initial state distribution d_0, with returns $G(\tau)$:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta, d_0}[G(\tau)] = \sum_\tau p_\theta(\tau) G(\tau)$$

where $p_\theta(\tau) = d_0(s_0) \prod_{t=0}^{T} \pi_\theta(a_t|s_t) p(s_{t+1}|s_t, a_t)$.

Goal: maximize expected return with gradient ascent:

$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$$

Policy Gradient (Episodic)

$$
\begin{aligned}
\nabla_\theta J(\theta) &= \nabla_\theta \mathbb{E}_{\tau\sim\pi_\theta,d_0}[G(\tau)] \\
&= \mathbb{E}_{\tau\sim\pi_\theta,d_0}\left[G(\tau)\nabla_\theta \log p_\theta(\tau)\right] && \text{see Proof (I)} \\
&= \mathbb{E}_{\tau\sim\pi_\theta,d_0}\left[\sum_{t=0}^{T} G(\tau)\nabla_\theta \log \pi_\theta(A_t|S_t)\right] && \text{see Proof (II)} \\
&= \mathbb{E}_{\tau\sim\pi_\theta,d_0}\left[\sum_{t=0}^{T} (\gamma^t G_t)\nabla_\theta \log \pi_\theta(A_t|S_t)\right] && \text{see Proof (III)}
\end{aligned}
$$

- Proof (I): log-likelihood trick
- Proof (II): policy gradient structure (env dynamics don't depend on θ)
- Proof (III): causality (future rewards don't depend on past actions)
- In practice γ^t is often omitted in episodic case

Policy Gradient Methods

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\sum_{t=0}^{T} \Psi_t \nabla_\theta \log \pi_\theta(A_t|S_t)\right] \quad \text{with } \Psi_t \text{ as learning signal}$$

$\Psi_t = G_t$	REINFORCE (MC return)
$= G_t - v_{\mathbf{w}}(S_t)$	REINFORCE with baseline†
$= \delta_t = R_{t+1} + \gamma v_{\mathbf{w}}(S_{t+1}) - v_{\mathbf{w}}(S_t)$	TD(0) actor-critic (*A2C*)†
$= \delta_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n v_{\mathbf{w}}(S_{t+n}) - v_{\mathbf{w}}(S_t)$	TD(n) actor-critic (*A2C*)†
$= \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$	TD(λ) / GAE (*PPO*)†
$= q_{\mathbf{w}}(S_t, A_t)$	Q-based actor-critic (*DDPG, SAC*)

Advantage estimators (†): $\hat{A}_t \approx A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t)$

Bias vs Variance in Policy Gradients

- Most practical methods use biased critics
  - TD, GAE, Q-functions
- In principle:
  - Compatible function approximation
  - allows unbiased gradients
- In practice:
  - rarely used in deep RL
  - too restrictive (linear, tied to policy)
- Deep RL prefers:

$$\text{low variance + expressive models} \;>\; \text{unbiased gradients}$$

Limitations of Actor-Critic

- On-policy updates
  - Data collected with current policy π_θ
  - Old data cannot be reused
  - $\Rightarrow$ sample inefficient
- Unstable policy updates
  - Distance in parameter space $\neq$ distance in policy space
  - Small change in parameters can drastically change π_θ
  - Bad update $\Rightarrow$ worse data $\Rightarrow$ collapse
- High variance
  - Even with baselines / GAE
  - Noisy gradient estimates

Off-Policy Policy Gradients

Off-Policy Challenge

- Data collected from behavior policy μ
- Want to evaluate / improve target policy π
- Mismatch:

$$a \sim \mu(\cdot|s) \quad \text{but want expectations under } \pi(\cdot|s)$$

- Leads to biased policy gradient estimates if ignored

Importance Sampling

- Correct distribution mismatch via reweighting:

$$\mathbb{E}_{a\sim\pi}[f(s,a)] = \mathbb{E}_{a\sim\mu}\left[\frac{\pi(a|s)}{\mu(a|s)}f(s,a)\right]$$

- Importance weight per action:

$$w = \frac{\pi(a|s)}{\mu(a|s)}$$

- Unbiased, but can have high variance

Off-Policy Policy Gradient

- Standard policy gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_\pi \left[\nabla_\theta \log \pi_\theta(a|s)\, G_t \right]$$

- Off-policy with importance sampling:

$$\nabla_\theta J(\theta) = \mathbb{E}_\mu \left[\frac{\pi_\theta(a|s)}{\mu(a|s)} \nabla_\theta \log \pi_\theta(a|s)\, G_t \right]$$

- Over full trajectories:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \mu} \left[\left(\prod_{t=0}^{T} \frac{\pi_\theta(a_t|s_t)}{\mu(a_t|s_t)} \right) \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) G(\tau) \right]$$

- Problem: high variance
  - product of many ratios $\Rightarrow$ can explode or vanish

Reducing Variance in Off-Policy Learning

- Per-decision importance sampling:

$$\prod_{t'=0}^{t} \frac{\pi(a_{t'}|s_{t'})}{\mu(a_{t'}|s_{t'})}$$

  - corrects mismatch only up to time t
  - avoids dependence on future actions
- Problem: importance weights $\Rightarrow$ high variance
- In practice:
  - constrain policy updates (stay close to on-policy: TRPO, PPO)
  - or avoid importance sampling entirely

Alternative: Replay-Based Off-Policy Learning

- Instead of correcting mismatch explicitly:
  - reuse past data (replay buffer)
  - learn from bootstrapped TD targets
- Result:
  - lower variance, more stable updates
  - widely used in deep RL (DQN, DDPG, SAC)
- Trade-off: introduces bias

Stable Updates

Unstable Policy Updates

- Policy gradient updates can drastically change the policy
- Why?
  - Small parameter change $\neq$ small change in action probabilities
  - Same parameter change can lead to very different policy changes
  - Near deterministic policies $\Rightarrow$ gradient estimates become unreliable
- Consequence:
  - Large updates $\Rightarrow$ worse policy $\Rightarrow$ worse data
  - Feedback loop leads to instability or collapse
- $\Rightarrow$ Control updates in policy space, not parameter space

Measuring Policy Change

- Measure how the action distribution changes
- Measure difference between policies with KL divergence:

$$D_{\mathrm{KL}}(\pi_{\theta_{\text{old}}} \,\|\, \pi_\theta) = \mathbb{E}_{s\sim d_{\pi_{\theta_{\text{old}}}}} \left[\sum_a \pi_{\theta_{\text{old}}}(a|s) \log \frac{\pi_{\theta_{\text{old}}}(a|s)}{\pi_\theta(a|s)} \right]$$

- Small KL $\Rightarrow$ similar action distributions

- Idea: Maximize return while limiting policy change:

$$\max_\theta \; J(\theta) \quad \text{s.t.} \quad D_{\mathrm{KL}}(\pi_{\theta_{\text{old}}} \,\|\, \pi_\theta) \leq \epsilon$$

Natural Policy Gradient (NPG)

- Standard gradient is steepest in parameter space, not in KL geometry
- Idea: take steepest ascent in policy space (KL), not parameter space
- Local approximation of KL:

$$D_{\mathrm{KL}}(\pi_\theta \,\|\, \pi_{\theta+\delta}) \approx \tfrac{1}{2}\delta^T F\, \delta$$

- F = Fisher information matrix (local curvature of KL)
- Natural gradient update:

$$\theta \leftarrow \theta + \alpha\, F^{-1} \nabla_\theta J(\theta)$$

- Interpretation:
  - rescales update to produce controlled change in policy

Peters and Schaal, 2008 (Amari 1998, Kakade 2002)

NPG Limitations

- Computationally expensive:
  - second-order method (Fisher matrix)
  - requires solving a linear system (e.g., conjugate gradient)
- Uses local KL geometry:
  - only a local approximation
  - no explicit constraint on policy change
- $\Rightarrow$ large, destructive updates still possible
- $\Rightarrow$ no guarantee of monotonic improvement

Trust Region Policy Optimization (TRPO)

- TRPO makes the KL constraint explicit and optimizes a surrogate objective:

$$\max_{\theta} \; \mathbb{E}_{(s_t,a_t)\sim\pi_{\theta_{\text{old}}}} \Bigg[\underbrace{\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}}_{r_t(\theta)} \underbrace{\hat{A}_t}_{\substack{\text{advantage estimate} \\ \text{under } \pi_{\theta_{\text{old}}}}} \Bigg] \quad \text{s.t.} \quad D_{\text{KL}}(\pi_{\theta_{\text{old}}} \,\|\, \pi_\theta) \leq \epsilon$$

Schulman et al., 2015

Why Use a Surrogate Objective?

- True objective:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)]$$

- Problem 1: cannot evaluate $J(\theta)$ directly
  - depends on trajectories from π_θ
  - $\Rightarrow$ requires new data after every update
- Problem 2: we only have data from $\pi_{\theta_{\text{old}}}$
  - $\mathbb{E}_{\pi_\theta}[\cdot] \neq \mathbb{E}_{\pi_{\theta_{\text{old}}}}[\cdot]$
  - $\Rightarrow$ distribution mismatch
- Problem 3: large updates break the approximation
  - gradient estimates only reliable locally
  - $\Rightarrow$ need to restrict policy changes
- $\Rightarrow$ optimize a surrogate objective + constrain policy change

Trust Region Policy Optimization (TRPO)

- TRPO makes the KL constraint explicit and optimizes a surrogate objective:

$$\max_{\theta} \; \mathbb{E}_{(s_t,a_t)\sim\pi_{\theta_{\text{old}}}} \Bigg[\underbrace{\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}}_{r_t(\theta)} \underbrace{\hat{A}_t}_{\substack{\text{advantage estimate} \\ \text{under } \pi_{\theta_{\text{old}}}}} \Bigg] \quad \text{s.t.} \quad D_{\text{KL}}(\pi_{\theta_{\text{old}}} \,\|\, \pi_\theta) \leq \epsilon$$

- Enforces small, reliable policy updates
- Trust region $\Rightarrow$ small KL $\Rightarrow$ surrogate approximates true objective
- Importance sampling $\Rightarrow$ reuse data from $\pi_{\theta_{\text{old}}}$
- Robust, but algorithmically complex and computationally expensive:
  - second-order approximation of KL (Fisher)
  - conjugate gradient and line search

Schulman et al., 2015

Proximal Policy Optimization (PPO)

- Replace hard KL constraint with a clipped surrogate objective:

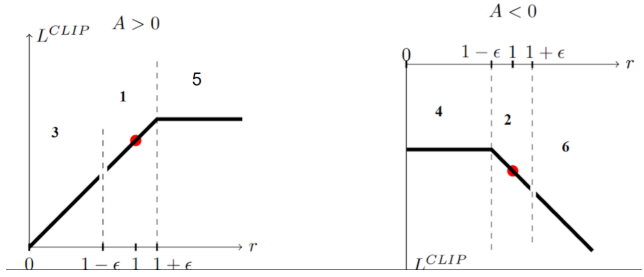
$$\max_{\theta} \ \mathbb{E}_{(s_t,a_t)\sim\pi_{\theta_{\text{old}}}}\left[\min\left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta),1-\epsilon,1+\epsilon)\hat{A}_t\right)\right]$$

$$\text{with } r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

- Limit how much the policy ratio can change
- Clip the policy ratio to limit destructive updates
- First-order optimization (no Fisher / second-order methods)

Schulman et al., 2017

PPO: Intuition of Clipping



	$\Delta\pi(a)$	$\hat{A}_t$	clipped	return of min	sign of objective	grad	effect on $\pi(a)$	
1	$\sim$	$+$	✗	$r_t\hat{A}_t$	$+$	✓	$\uparrow$	increase probability
2	$\sim$	$-$	✗	$r_t\hat{A}_t$	$-$	✓	$\downarrow$	decrease probability
3	$\downarrow$	$+$	✗	$r_t\hat{A}_t$	$+$	✓	$\uparrow$	corrects previous decrease
4	$\downarrow$	$-$	✓	$(1-\epsilon)\hat{A}_t$	$-$	✗	$-$	already decreased a lot
5	$\uparrow$	$+$	✓	$(1+\epsilon)\hat{A}_t$	$+$	✗	$-$	already increased a lot
6	$\uparrow$	$-$	✗	$r_t\hat{A}_t$	$-$	✓	$\downarrow$	corrects previous increase

PPO: Data Efficiency

- Data collected with $\pi_{\theta_{\text{old}}}$
- In vanilla policy gradient:
  - one update per batch
  - data quickly becomes off-policy
- In PPO:
  - clipping keeps π_θ close to $\pi_{\theta_{\text{old}}}$
  - updates remain approximately on-policy
- Result:
  - multiple epochs over the same batch
  - improved sample efficiency

NPG, TRPO, PPO at a Glance

Method	On/Off policy	Constraint	Key idea
NPG	on-policy	implicit KL	natural gradient
TRPO	on-policy	hard KL	trust region
PPO	on-policy	soft KL	clipping

- TRPO/PPO use old data but remain on-policy
  - data comes from $\pi_{\theta_{\text{old}}}$ and updates stay close to it (trust region)
- PPO generally uses GAE advantages
- Limited updates $\Rightarrow$ risk of premature convergence
  - $\Rightarrow$ entropy regularization helps

Continuous Control

Continuous Actions

- Value-based methods must solve $a^*(s) = \operatorname{argmax}_a q(s, a)$ which is non-trivial in continuous action spaces
- Policy-based methods avoid this inner maximization
- Same policy-gradient machinery works for discrete and continuous actions
- Main challenge shifts to policy parameterization and exploration
  - Learn parameters of a probability density function (e.g. Gaussian) rather than discrete probabilities
  - Exploration can be challenging

Example: Gaussian Policy

A common stochastic policy for continuous control is

$$A_t \sim \pi_\theta(\cdot \mid S_t) = \mathcal{N}(\mu_\theta(S_t), \Sigma_\theta(S_t))$$

(μ_θ is the mean – not to be confused with the behavior policy μ)

- Actor outputs the mean $\mu_\theta(s)$; variance $\Sigma_\theta(s)$ can be fixed or learned
- The log-policy gradient (with fixed variance) becomes

$$\nabla_\theta \log \pi_\theta(A_t \mid S_t) = \frac{A_t - \mu_\theta(S_t)}{\sigma^2} \nabla_\theta \mu_\theta(S_t)$$

Policy Gradient with Gaussian Actor

Starting from REINFORCE with a baseline,

$$\theta \leftarrow \theta + \alpha\big(G_t - v_{\mathbf{w}}(S_t)\big)\nabla_\theta \log \pi_\theta(A_t \mid S_t)$$

we get

$$\theta \leftarrow \theta + \alpha\big(G_t - v_{\mathbf{w}}(S_t)\big)\frac{A_t - \mu_\theta(S_t)}{\sigma^2}\nabla_\theta \mu_\theta(S_t)$$

- Positive advantage: move mean toward sampled action
- Negative advantage: move mean away from sampled action
- This is the standard actor-critic template for continuous stochastic policies

Deterministic Policy Gradient (DPG)

- Policy gradients rely on sampling $\Rightarrow$ high variance
- Critic is used indirectly (via advantage estimates)
- If $Q(s, a)$ generalizes well, we can optimize actions directly
  - Learn action-value function $Q_{\mathbf{w}}(s, a) \approx Q^{\mu}(s, a)$
  - Define deterministic policy: $A_t = \mu_\theta(S_t)$
  - Improve policy via gradient ascent on the value:

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\nabla_a Q_{\mathbf{w}}(s, a)\big|_{a=\mu_\theta(s)} \nabla_\theta \mu_\theta(s)\right]$$

- Alternating evaluation and improvement $\Rightarrow$ policy iteration

---

Silver et al., 2014 (Werbos 1990, Prokhorov & Wunsch 1997, Hasselt & Wiering 2007)

Deep Deterministic Policy Gradient (DDPG)

- Make deterministic policy gradient work in practice
- Off-policy learning with replay buffer
  - reuse past experience $\Rightarrow$ sample efficient
- Use target networks for stability
  - slowly updated copies of actor and critic
- Critic update with TD:

$$\min_{\mathbf{w}} \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\Big[\big(Q_{\mathbf{w}}(s,a)-y\big)^2\Big] \quad \text{where} \quad y = r + \gamma Q_{\mathbf{w}^-}(s', \mu_{\theta^-}(s'))$$

- Actor update with deterministic policy gradient:

$$\max_{\theta} \mathbb{E}_{s\sim\mathcal{D}}\big[Q_{\mathbf{w}}(s, \mu_\theta(s))\big]$$

Lillicrap et al., 2015

DDPG Algorithm

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ.
Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$
Initialize replay buffer R
for episode = 1, M **do**
  Initialize a random process $\mathcal{N}$ for action exploration
  Receive initial observation state s_1
  for t = 1, T **do**
    Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise
    Execute action a_t and observe reward r_t and observe new state s_{t+1}
    Store transition (s_t, a_t, r_t, s_{t+1}) in R
    Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R
    Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$
    Update critic by minimizing the loss: $L = \frac{1}{N}\sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$
    Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

    Update the target networks:

$$\theta^{Q'} \leftarrow \tau\theta^Q + (1-\tau)\theta^{Q'}$$
$$\theta^{\mu'} \leftarrow \tau\theta^\mu + (1-\tau)\theta^{\mu'}$$

  end for
end for

DQN vs DDPG

- Both follow the same core idea:
  - off-policy learning with replay buffer
  - bootstrapped TD targets

$$y = r + \gamma Q(s', a')$$

  - target networks for stability
- Same critic, different actor:

DQN: $a = \arg\max_{a'} Q(s, a')$ (explicit maximization)

DDPG: $a = \mu_\theta(s)$ (learned maximizer)

- DDPG = DQN + learned argmax for continuous actions

Limitations of DDPG

- Overestimation bias in Q-values
- Unstable training
  - errors in critic $\Rightarrow$ bad actor updates
- Deterministic policy
  - exploration must be added manually (noise)

$\Rightarrow$ TD3: stabilize critic and policy updates

Twin Delayed DDPG (TD3)

- Addresses instability of DDPG
- Clipped Double Q-learning
  - use two critics and take $\min(Q_1, Q_2)$
  - reduces overestimation bias
- Delayed policy updates
  - update actor less frequently than critic
- Target policy smoothing
  - add noise to target actions
  - prevents overfitting to sharp Q peaks

Algorithm 1 TD3

Initialize critic networks Q_{θ_1}, Q_{θ_2}, and actor network π_ϕ with random parameters θ_1, θ_2, ϕ
Initialize target networks $\theta_1' \leftarrow \theta_1$, $\theta_2' \leftarrow \theta_2$, $\phi' \leftarrow \phi$
Initialize replay buffer $\mathcal{B}$
for $t = 1$ **to** T **do**
  Select action with exploration noise $a \sim \pi_\phi(s) + \epsilon$, $\epsilon \sim \mathcal{N}(0, \sigma)$ and observe reward r and new state s'
  Store transition tuple (s, a, r, s') in $\mathcal{B}$

  Sample mini-batch of N transitions (s, a, r, s') from $\mathcal{B}$
  $\tilde{a} \leftarrow \pi_{\phi'}(s') + \epsilon, \quad \epsilon \sim \text{clip}(\mathcal{N}(0, \tilde{\sigma}), -c, c)$
  $y \leftarrow r + \gamma \min_{i=1,2} Q_{\theta_i'}(s', \tilde{a})$
  Update critics $\theta_i \leftarrow \text{argmin}_{\theta_i} N^{-1} \sum (y - Q_{\theta_i}(s, a))^2$
  if t mod d **then**
    Update ϕ by the deterministic policy gradient:
    $\nabla_\phi J(\phi) = N^{-1} \sum \nabla_a Q_{\theta_1}(s, a)|_{a=\pi_\phi(s)} \nabla_\phi \pi_\phi(s)$
    Update target networks:
    $\theta_i' \leftarrow \tau\theta_i + (1 - \tau)\theta_i'$
    $\phi' \leftarrow \tau\phi + (1 - \tau)\phi'$
  end if
end for

Fujimoto et al., 2018

From Deterministic to Stochastic Policies

- DDPG / TD3 use deterministic policies
- Can collapse early to suboptimal actions
- Exploration via added noise is often brittle
- Desideratum: high reward **and** appropriate uncertainty
- Can we learn stochastic policies directly?

Maximum Entropy Reinforcement Learning

- Standard RL objective:

$$\max_{\pi} \mathbb{E}_{\pi}\left[\sum_{t=0}^{T} \gamma^t R_{t+1}\right]$$

- Maximum entropy RL adds an entropy bonus:

$$\max_{\pi} \mathbb{E}_{\pi}\left[\sum_{t=0}^{T} \gamma^t \big(R_{t+1} + \alpha \mathcal{H}(\pi(\cdot|S_t))\big)\right]$$

- Entropy term:

$$\mathcal{H}(\pi(\cdot|s)) = -\mathbb{E}_{a\sim\pi(\cdot|s)}[\log \pi(a|s)]$$

- Encourages:
  - sustained exploration
  - robustness to errors in value estimates
  - smoother, less brittle updates

Soft Actor-Critic (SAC)

- Off-policy actor-critic with maximum entropy objective
- Learns a stochastic policy $\pi_\theta(a|s)$
- Uses:
  - replay buffer (off-policy)
  - target networks for stability
  - double Q-learning (two critics)
- Critic update with TD:

$$\min_{\mathbf{w}} \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\Big[\big(Q_{\mathbf{w}}(s,a) - y\big)^2\Big]$$

$$y = r + \gamma\, \mathbb{E}_{a'\sim\pi_\theta}\Big[Q_{\mathbf{w}^-}(s',a') - \alpha \log \pi_\theta(a'|s')\Big]$$

- Actor update:

$$\max_{\theta} \mathbb{E}_{s\sim\mathcal{D},\, a\sim\pi_\theta}\big[Q_{\mathbf{w}}(s,a) - \alpha \log \pi_\theta(a|s)\big]$$

Haarnoja et al., 2018

DDPG / TD3 vs SAC

- DDPG / TD3:
  - deterministic policy $a = \mu(s)$
  - exploration via added noise
  - continuous action spaces
  - efficient when optimal policy is near-deterministic
- SAC:
  - stochastic policy $\pi(a|s)$
  - exploration via entropy objective
  - continuous (standard), discrete (variant)
  - preferred when exploration and robustness matter

Where Do PPO, TD3, and SAC Fit in Practice?

	PPO	TD3	SAC
Policy type	stochastic	deterministic	stochastic
Data regime	on-policy	off-policy	off-policy
Best fit	stable large-scale training	clean continuous-control baseline	robust, sample-efficient control
Use cases	simulation, LLMs, games	benchmark control, precise actuation	robotics, real-world control
Main trade-off	data hungry	weaker exploration	more moving parts

Method choice is a systems decision, not just an algorithm decision.